

SISTEMA DE VIGILANCIA

INTEGRANTES:

- CARLOS NICOLAS MORALES CUELLAR (23190329)
- JOSE MARTIN RAMOS CALZADA (231903417)
- ROJAS CRISOSTOMO RONALD ENRIQUE(23190346)



INTRODUCCIÓN

En el marco del desarrollo de soluciones tecnológicas aplicadas a la seguridad y al Internet de las Cosas (IoT), se presenta un sistema integrado que combina hardware de bajo costo con algoritmos de análisis en software. El proyecto consiste en una plataforma de vigilancia inteligente basada en el raspberry pi zero 2w, módulo ESP32-CAM (momentaneamente), complementada con un sensor de proximidad infrarrojo pasivo (PIR) y una interfaz de control que permite al usuario ejecutar funciones específicas como la captura de fotografías y el encendido del flash de la cámara.



PROBLEMÁTICA

La inseguridad ciudadana en Perú, marcada por el constante aumento de delitos como robos y violencia, es un desafío social y político crucial que afecta la calidad de vida y limita el desarrollo económico al disminuir la confianza pública; ante esta realidad, se plantea la necesidad de implementar soluciones tecnológicas accesibles y eficientes, como dispositivos de bajo costo con algoritmos de análisis y comunicación en tiempo real, para prevenir y disuadir actos delictivos, fortaleciendo la seguridad doméstica y comunitaria, especialmente en zonas con recursos limitados para la vigilancia tradicional.



COMPARACIÓN CON OTRAS CÁMARAS DEL MERCADO

Aunque una cámara comercial está empaquetada para el usuario final, nuestra arquitectura ofrece ventajas técnicas inmensas desde la perspectiva de la ingeniería, el control de redes y la ciberseguridad.

1

Transparencia y Observabilidad (Caja Blanca):

A diferencia de las cámaras comerciales que operan como "cajas negras" sin visibilidad de metadatos o telemetría, esta arquitectura garantiza observabilidad total. El diseño desde cero permite una inspección profunda (DPI) con Wireshark, eliminando el tráfico oculto y facilitando la auditoría precisa de cabeceras, tiempos de respuesta y secuencias TCP.



COMPARACIÓN CON OTRAS CÁMARAS DEL MERCADO

2

Independencia y seguridad:

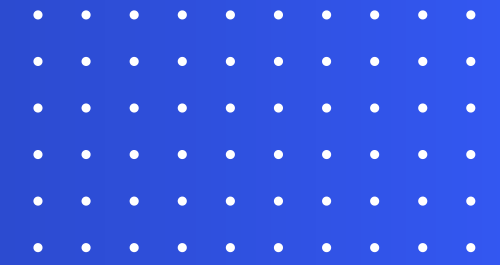
A diferencia de las cámaras comerciales que envían video constantemente a servidores externos (lo que satura la capacidad de subida de tu internet), nuestro sistema procesa todo en casa. Solo enviamos datos cuando el sensor detecta movimiento (el "disparador"), lo que ahorra energía y evita saturar tu conexión.

Independencia y seguridad: No dependemos de una aplicación específica. Usamos un servidor "puente" que conecta las cámaras con Telegram de forma segura. Tus cámaras nunca se conectan directamente a internet, por lo que están mejor protegidas. Además, si en el futuro quieres cambiar la forma de recibir alertas, solo necesitas ajustar este "puente" sin tener que reconfigurar las cámaras.



ANALISIS DE LAS CAPAS MODELO OSI

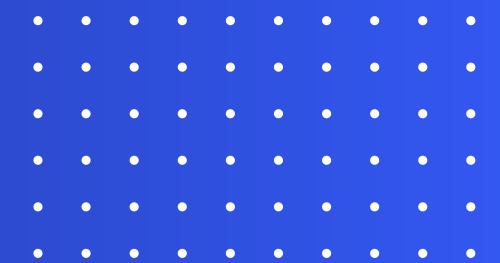
CAPA DE APLICACIÓN



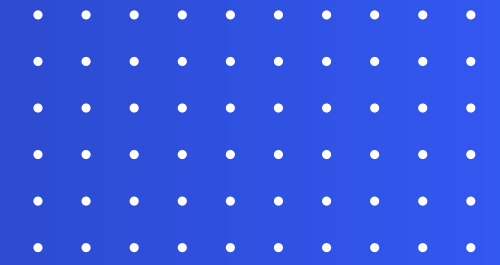
La capa de aplicación contiene la lógica principal del sistema de vigilancia. La Raspberry Pi captura imágenes mediante la cámara conectada por CSI o USB y las envía al servidor Flask mediante solicitudes HTTP POST. El servidor procesa la información y la reenvía a Telegram mediante la Bot API para notificar al usuario.

Protocolos y tecnologías utilizadas

- HTTP/1.1
- HTTPS
- REST
- Telegram Bot API



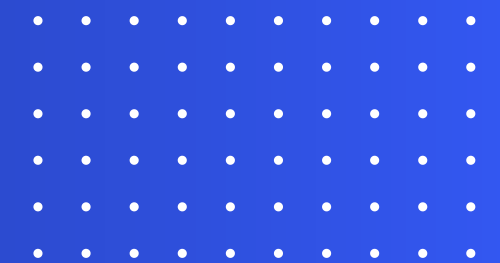
CAPA DE PRESENTACIÓN



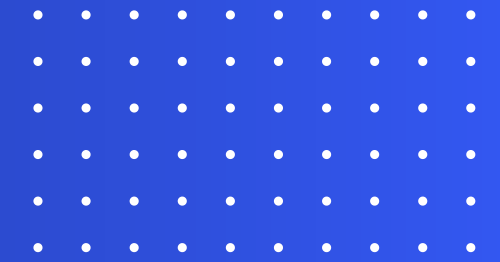
La capa de presentación se encarga de transformar y representar la información antes de su transmisión. Las imágenes capturadas por la cámara conectada a la Raspberry Pi son comprimidas en formato JPEG para reducir su tamaño. Posteriormente, estas imágenes son encapsuladas mediante multipart/form-data para ser enviadas al servidor Flask. Además, la comunicación con Telegram utiliza TLS 1.3 para proteger la información transmitida.

Protocolos y tecnologías utilizadas

- JPEG
- multipart/form-data
- TLS1.3



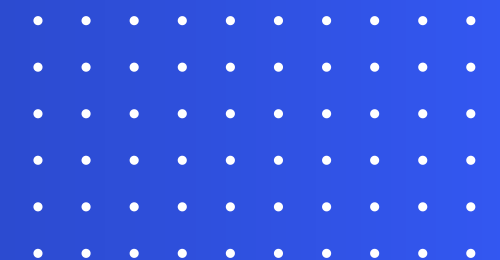
CAPA DE SESIÓN



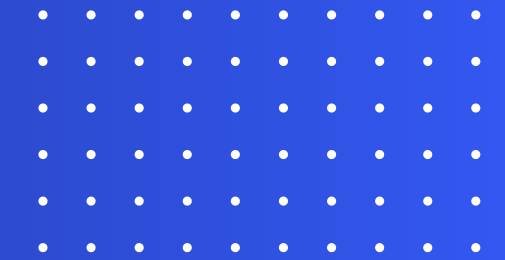
La capa de sesión administra el establecimiento, mantenimiento y finalización de las comunicaciones entre los diferentes componentes del sistema. En la red local, cada captura de imagen se realiza mediante solicitudes HTTP independientes (stateless). Hacia Telegram, el servidor Flask utiliza long polling para mantener una comunicación continua y recibir comandos en tiempo real.

Protocolos y tecnologías utilizadas

- HTTP/1.1
- HTTP Keep-Alive
- Long Polling
- Sesiones TCP semi-persistente
-



CAPA DE TRANSPORTE



La capa de transporte es responsable de proporcionar una comunicación extremo a extremo confiable entre las aplicaciones que participan en el sistema de vigilancia IoT. Su función principal es segmentar los datos, controlar el flujo de información, detectar errores y garantizar que los datos lleguen completos y en el orden correcto al destino.

Protocolos y tecnologías utilizadas
TCP (Transmission Control Protocol)

3825	39.585809400	192.168.0.12	192.168.0.8	TCP
3826	39.595376100	192.168.0.12	192.168.0.8	HTTP
3827	39.595417200	192.168.0.8	192.168.0.12	TCP
3828	39.595467500	192.168.0.8	192.168.0.12	TCP
3829	39.597596500	192.168.0.12	192.168.0.8	TCP
3831	39.627605000	192.168.0.8	192.168.0.12	TCP

```
226 5001 → 50423 [PSH, ACK] Seq=1 Ack=156 Win=64128 Len=172 [TCP PDU reassembled in 3826]
60 53130 → 5000 [ACK] Seq=4471 Ack=177 Win=64128 Len=0
60 HTTP/1.1 200 OK (text/html)
54 50423 → 5001 [ACK] Seq=156 Ack=176 Win=262400 Len=0
54 50423 → 5001 [FIN, ACK] Seq=156 Ack=176 Win=262400 Len=0
60 5001 → 50423 [ACK] Seq=176 Ack=157 Win=64128 Len=0
54 50024 → 22 [ACK] Seq=1 Ack=273 Win=1025 Len=0
```



CAPA DE RED



La capa de red es responsable del direccionamiento lógico y del encaminamiento de los paquetes entre los dispositivos de la arquitectura IoT. Su función es garantizar que la información enviada desde un origen llegue al destino correcto mediante direcciones IP, independientemente del medio físico utilizado.

Protocolos y tecnologías utilizadas

- IPv4
- ICMP
- NAT/PAT (salida hacia Internet)

```
C:\Windows\system32>ipconfig
```

```
Configuración IP de Windows
```

```
Adaptador de Ethernet Ethernet:
```

```
Sufijo DNS específico para la conexión. . . :  
Dirección IPv6 . . . . . : 2800:200:e800:3a9a:cd3a:bc3b:2bf9:be35  
Dirección IPv6 temporal. . . . . : 2800:200:e800:3a9a:4cf5:5bb7:ce22:6e3f  
Vínculo: dirección IPv6 local. . . : fe80::14ff:9d39:7177:3283%18  
Dirección IPv4. . . . . : 192.168.0.8  
Máscara de subred . . . . . : 255.255.255.0  
Puerta de enlace predeterminada . . . . : fe80::a239:eeff:fe96:f6fc%18  
                                              192.168.0.1
```

```
pi@raspberrypi:~$ ifconfig
```

```
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
```

```
inet 127.0.0.1 netmask 255.0.0.0
```

```
inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

```
loop txqueuelen 1000 (Local Loopback)
```

```
RX packets 17 bytes 2722 (2.6 KiB)
```

```
RX errors 0 dropped 0 overruns 0 frame 0
```

```
TX packets 17 bytes 2722 (2.6 KiB)
```

```
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
```

```
inet 192.168.0.12 netmask 255.255.255.0 broadcast 192.168.0.255
```

```
inet6 fe80::8aa2:9eff:fe5a:fc5e prefixlen 64 scopeid 0x20<link>
```

```
inet6 2800:200:e800:3a9a:8aa2:9eff:fe5a:fc5e prefixlen 64 scopeid 0x0<global>
```

```
ether 88:a2:9e:5a:fc:5e txqueuelen 1000 (Ethernet)
```

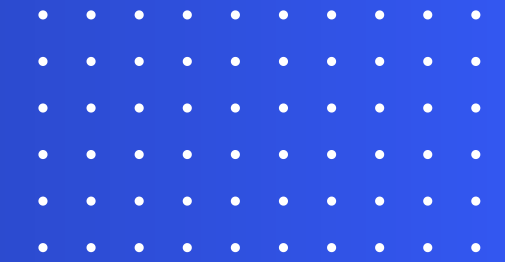
```
RX packets 73415 bytes 4504399 (4.2 MiB)
```

```
RX errors 0 dropped 0 overruns 0 frame 0
```

```
TX packets 69042 bytes 3814584 (3.6 MiB)
```

```
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

CAPA DE ENLACE DE DATOS



La capa de enlace de datos es responsable de la comunicación entre dispositivos dentro de la misma red local. Su función principal es encapsular los datos en tramas, controlar el acceso al medio inalámbrico y permitir la identificación física de los equipos mediante direcciones MAC.

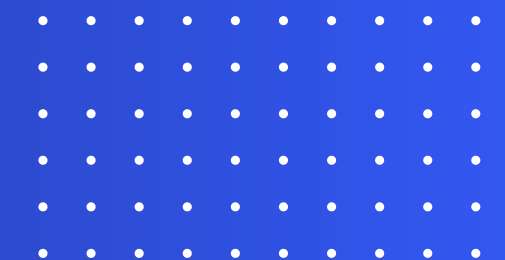
Protocolos y tecnologías utilizadas

- IEEE 802.11n (Wi-Fi)
- CSMA/CA (Control de acceso al medio)
- ARP (Address Resolution Protocol)
- Direcciones MAC

```
C:\Windows\system32>arp -a

Interfaz: 192.168.56.1 --- 0xc
Dirección de Internet      Dirección física      Tipo
224.0.0.22                01-00-5e-00-00-16    estático
224.0.0.251               01-00-5e-00-00-fb    estático
239.255.255.250           01-00-5e-7f-ff-fa    estático

Interfaz: 192.168.0.8 --- 0x12
Dirección de Internet      Dirección física      Tipo
192.168.0.1               a0-39-ee-96-f6-fc    dinámico
192.168.0.12              88-a2-9e-5a-fc-5e    dinámico
224.0.0.22                01-00-5e-00-00-16    estático
224.0.0.251               01-00-5e-00-00-fb    estático
239.255.255.250           01-00-5e-7f-ff-fa    estático
```



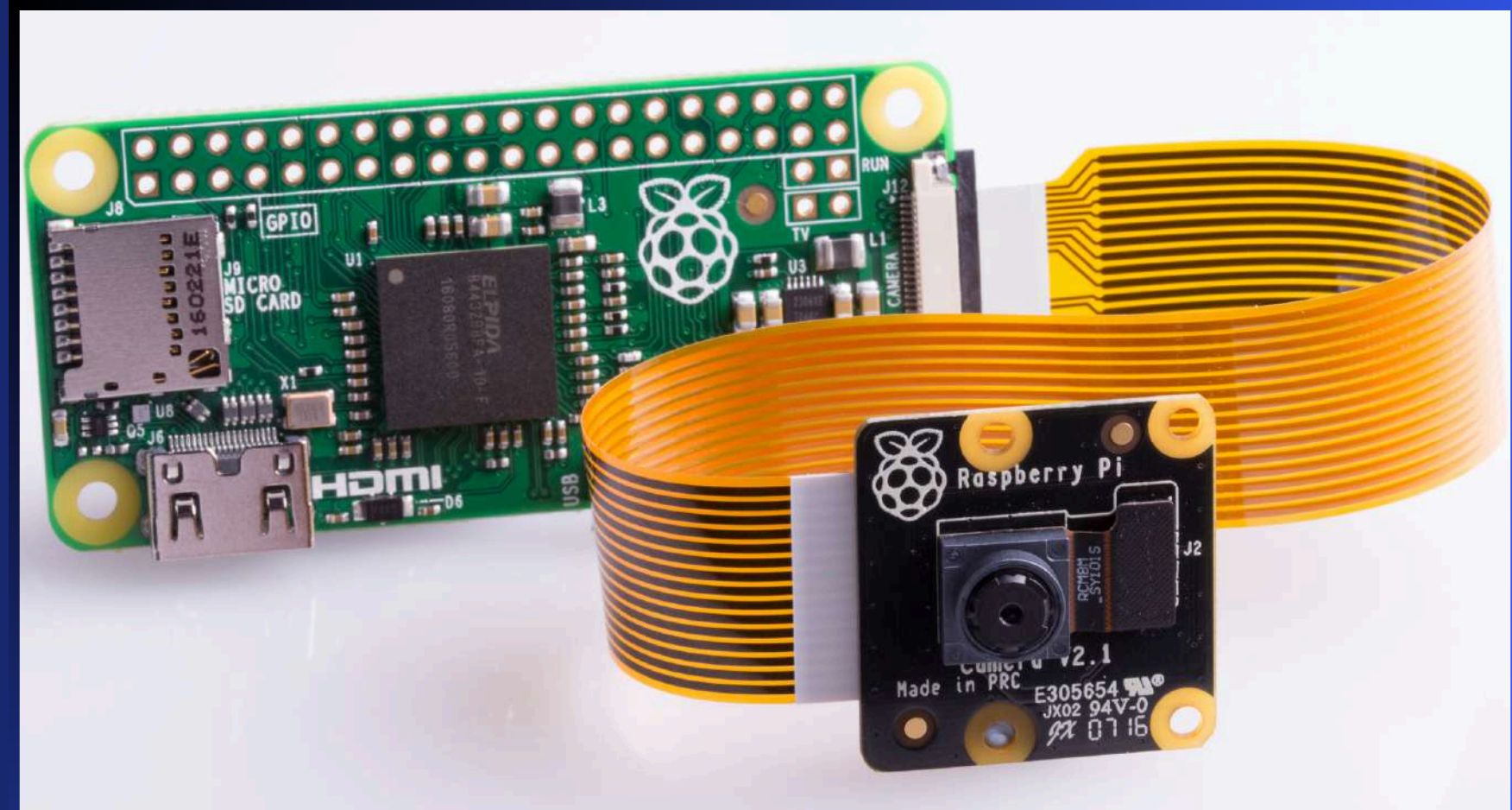
CAPA FÍSICA

Componentes físicos utilizados

- Raspberry Pi Zero 2W
- Cámara CSI conectada mediante interfaz MIPI CSI-2
- Tarjeta Wi-Fi IEEE 802.11n integrada
- PC con servidor Flask
- Medio inalámbrico de 2.4 GHz
- Pir sensor hc sr501

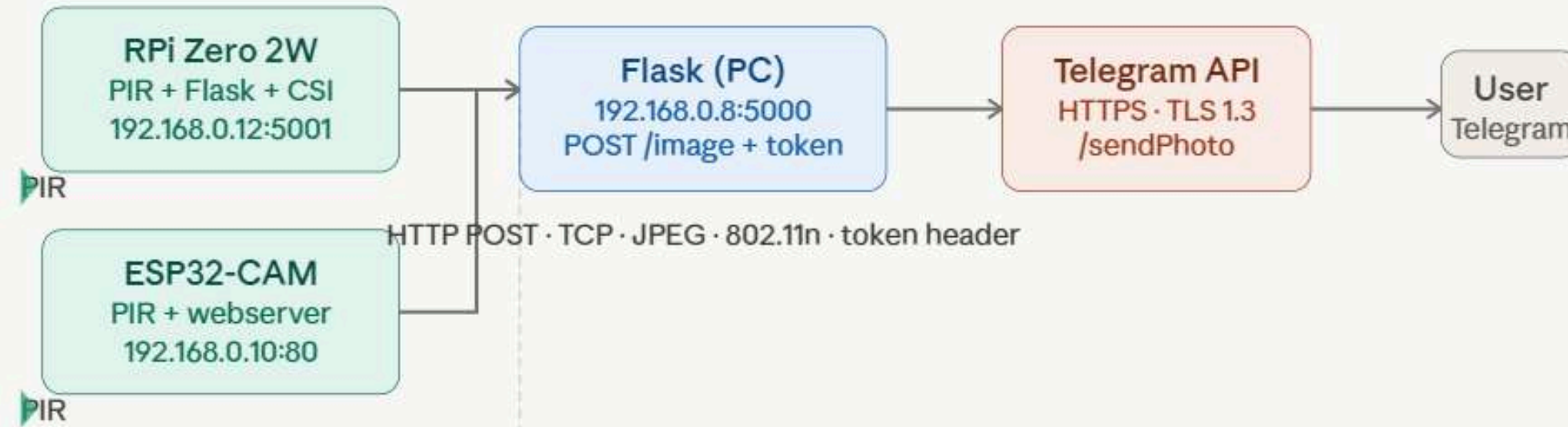
Tecnologías de Capa Física

- IEEE 802.11n (Wi-Fi 4)
- OFDM (Orthogonal Frequency Division Multiplexing)
- Banda de 2.4 GHz
- Interfaz MIPI CSI-2



SISTEMA DE MONITOREO PIR-TELEGRAMA

Flujo 1 — Sensor → Usuario (trigger autónomo por PIR integrado)



Flujo 2 — Usuario → Sensor (comando dedicado con routing explícito)



TELEGRAM BOT API

Es una interfaz basada en HTTP/HTTPS que facilita la comunicación entre una aplicación (bot) y los servidores de Telegram.

CREACIÓN DEL BOT

- Se inicia con **@BotFather**, el bot oficial de Telegram para registrar nuevos bots.
- Obtienes un token único que autentica tu aplicación ante los servidores de Telegram.

COMUNICACIÓN VÍA HTTP

Todas las peticiones se hacen mediante HTTPS a la URL:

<https://api.telegram.org/bot<token>/<method>>

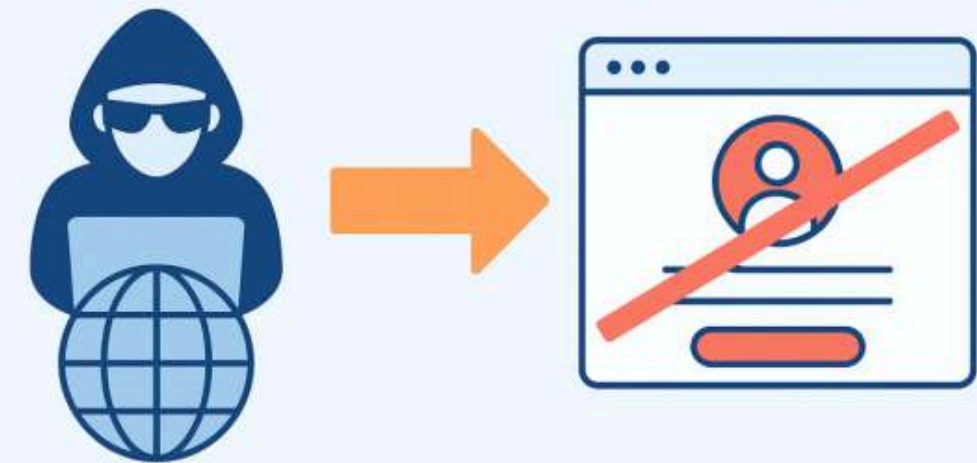
Soporta los métodos HTTP principales: GET y POST.

Podemos enviar parámetros de varias formas

- Query string (en la URL)
- application/x-www-form-urlencoded
- application/json
- multipart/form-data (para subir archivos como fotos, documentos, etc.)



INSECURE DIRECT OBJECT REFERENCE (IDOR)



TELEGRAM BOT API

CONCEPTUALMENTE SE ENVÍA ALGO PARECIDO A:

POST /sendPhoto HTTP/1.1

Content-Type: multipart/form-data

--boundary

Content-Disposition: form-data; name="chat_id"

123456789

--boundary

Content-Disposition: form-data; name="caption"

Captura detectada

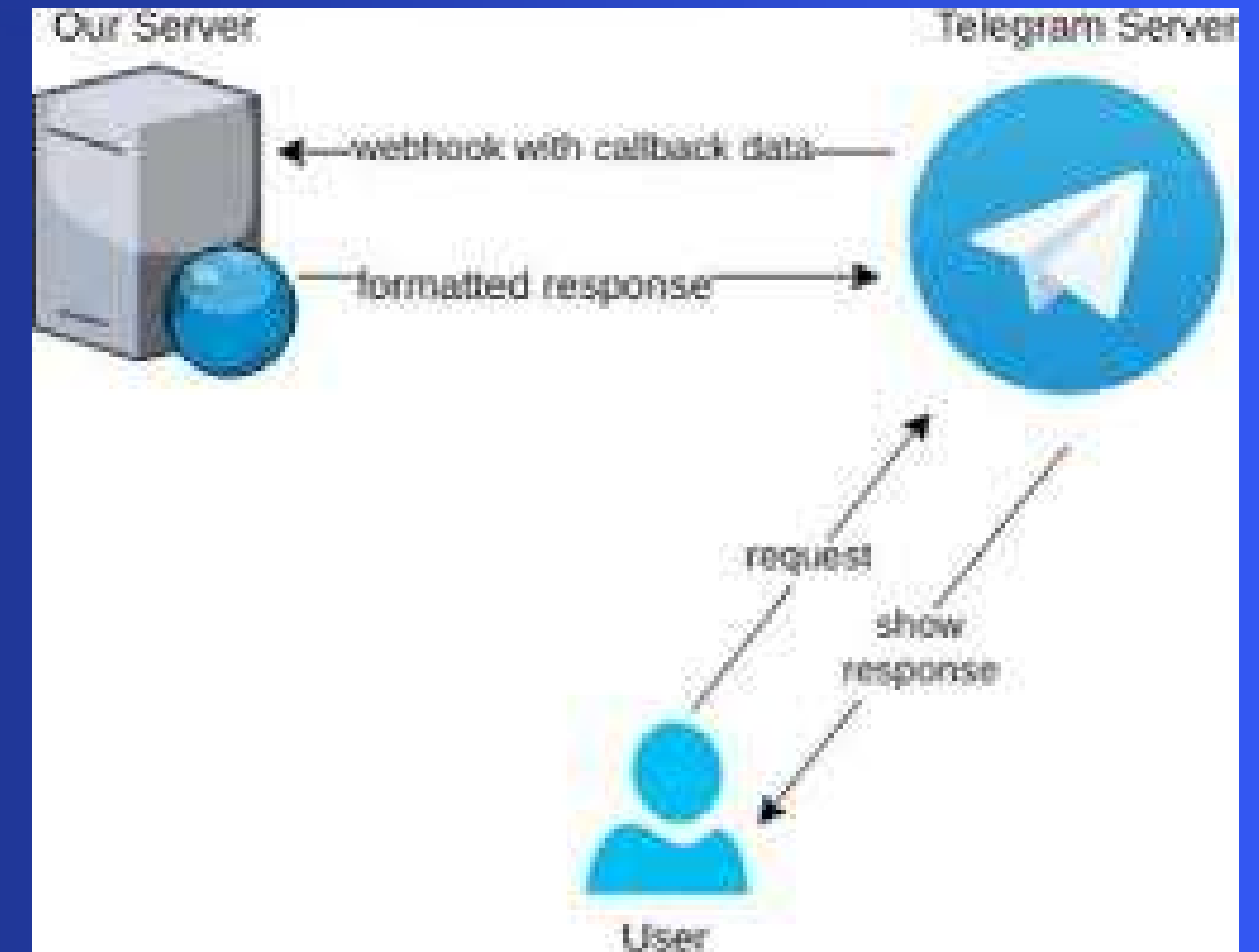
--boundary

Content-Disposition: form-data; name="photo"; filename="captura.jpg"

Content-Type: image/jpeg

...bytes de la imagen...

--boundary--

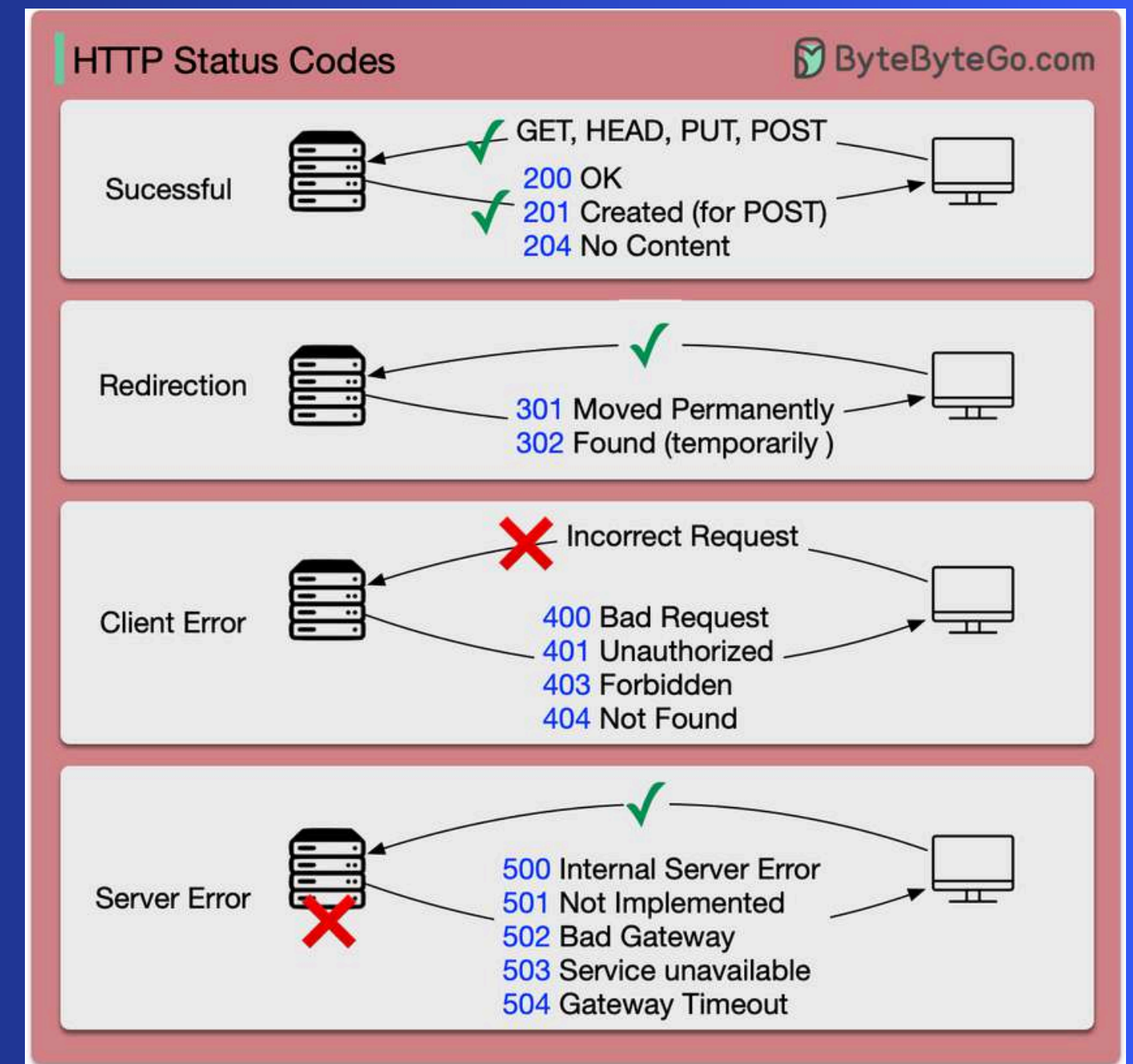


FLASK → TELEGRAM

GENERACIÓN DE PETICIÓN

```
r = requests.post(url, files=files, data=data, timeout=5)
r.raise_for_status()
```

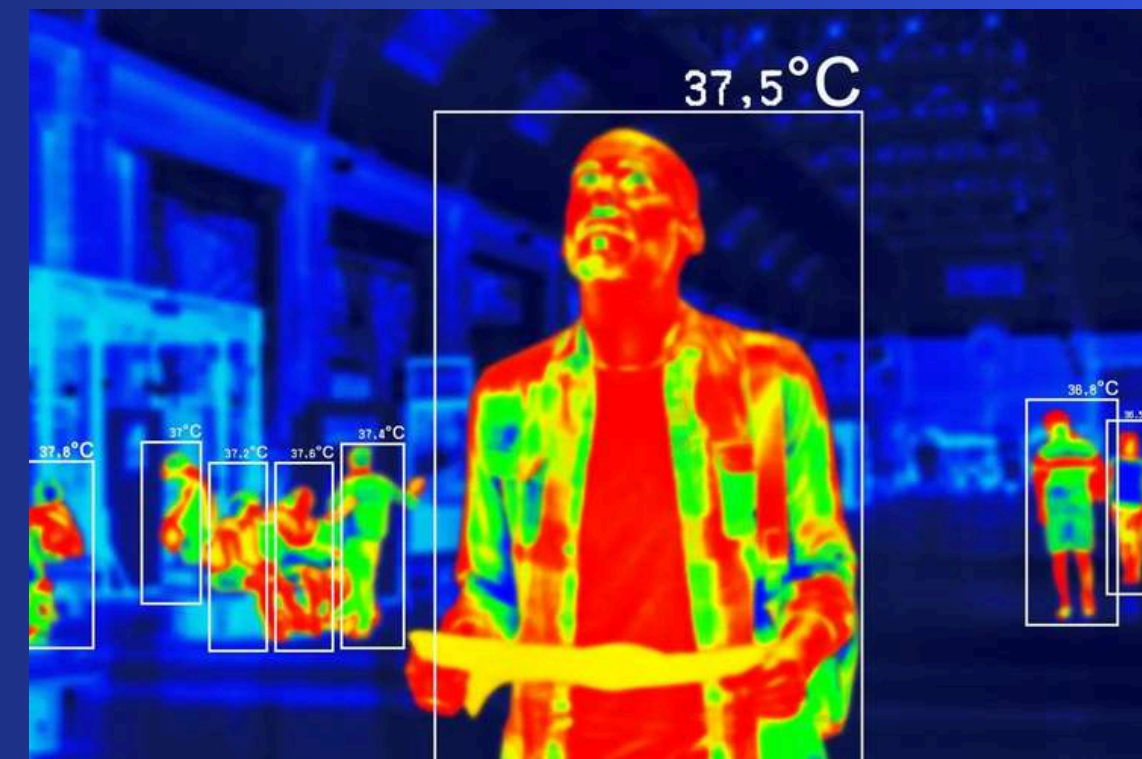
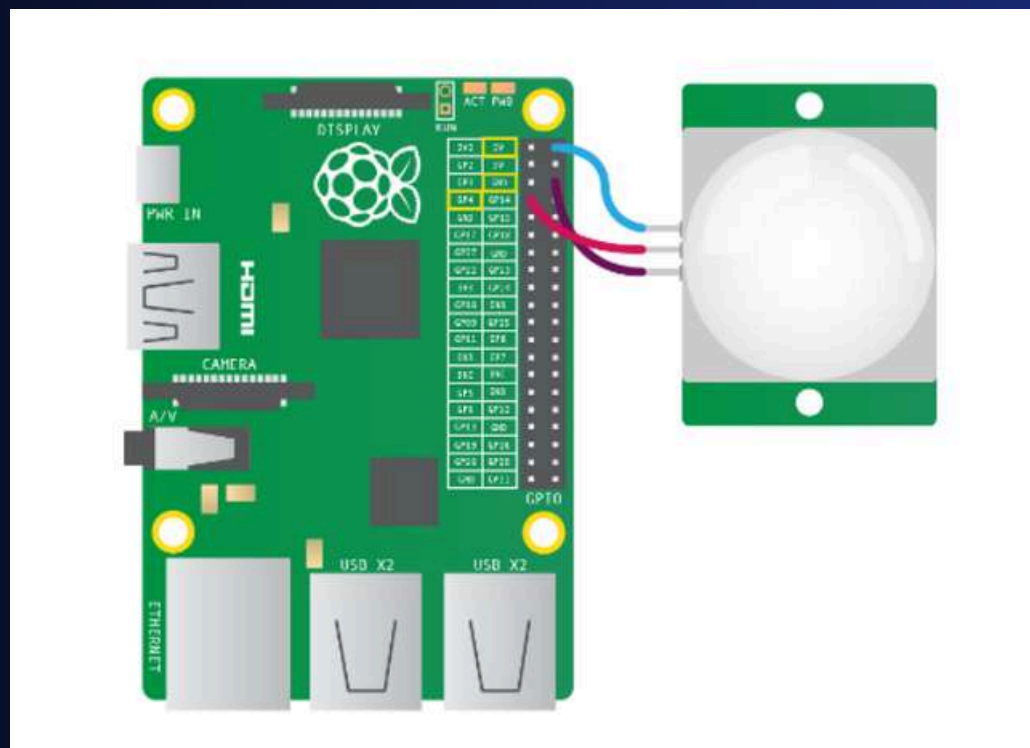
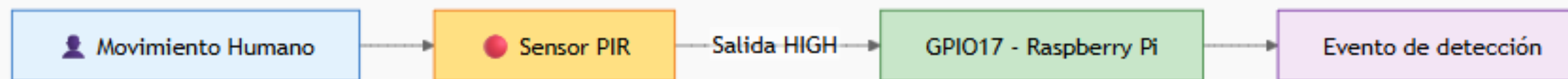
- 200 OK → No hace nada y el programa continúa.
- 201, 202, etc. → Tampoco genera error.
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

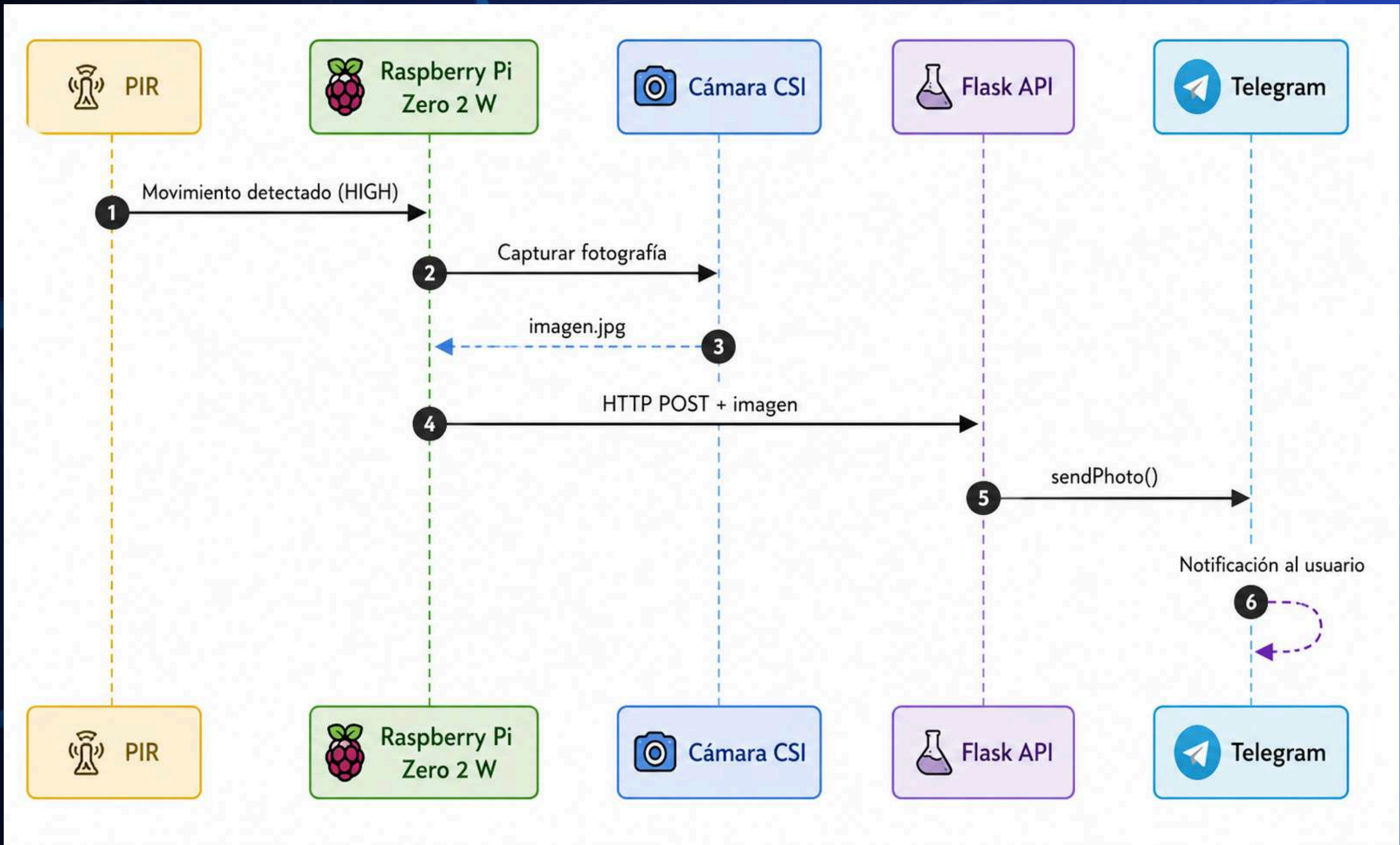


PIR + RPI + CAM CSI

¿Qué es un sensor PIR?

Un PIR (Passive Infrared Sensor) es un sensor electrónico que detecta variaciones en la radiación infrarroja emitida por cuerpos calientes (personas, animales u objetos con temperatura superior al entorno).





Drive



<https://drive.google.com/drive/folders/1uGrhmf53hAXfMhQ4NijYz9XCG8cgZpfo>

En funcionamiento





THANK YOU

